Engage: start here

This guide tells you what to read and in what order, which files make up the product, which files you need to deploy it, how to deploy it step by step, and how to check that it works.

Where things stand (13 October 2026). Engage is built as a working prototype and tested. It is **not deployed anywhere yet**. Deploying needs your hosting account. Everything below is written so you can do it without an engineer, and so an engineer can check your work.

---

1. What to read, in order

The 30 minute path (understand the product)

#	Open this	What it tells you	Time
1	`README.md`	What Engage is, how to run it, what it includes	5 min
2	`docs/engage-product-plan-report.pdf` (17 pages)	The product explained in 9 steps with screenshots, the plan, the risks	20 min
3	`docs/STATUS.md`	Exactly what is built, what is tested, what is not	5 min

The full path (understand everything, in order)

#	Open this	Pages	What it answers
1	`README.md`	1	What is it, how do I run it
2	`docs/engage-product-plan-report.pdf`	17	What does it look like, who is it for, what is the plan
3	`docs/Engage-PRD.pdf`	13	Exactly what must the product do. Every requirement has a status (Built, Planned, Later)
4	`docs/design/Engage-UX-UI-Design.pdf`	16	How it should look and behave. Design system, accessibility numbers, usability test plan
5	The clickable prototype (see below)		Try the mini window, recall check, presenting mode and coach summary yourself
6	`docs/architecture/Engage-Technical-Architecture.pdf`	15	How it is built now, how it should be built next, and why
7	`docs/architecture/Engage-Engineering-Plan.pdf`	8	What to build next, in what order, how long it takes
8	`docs/STATUS.md`	1	What is done and not done, against the plan
9	`docs/deploy.md`	1	The short deployment guide (this guide has the long version)
10	`docs/START-HERE.md` (this file)		The map

The clickable prototype. Two ways to open it:

- Open `docs/design/prototype/index.html` in any browser. It works offline.
- Or use the private page: https://claude.ai/artifact/T8dsvRAvFJXMwpUu2vkjyh (only you can open it until you share it).

Background documents (optional)

File	What it is
`docs/phase1-spec.md`	The first version of the Phase 1 scope. Replaced by the PRD
`docs/product-plan.md`	The first product plan. Replaced by the report and the engineering plan
`docs/PRD.md`, `docs/architecture/*.md`	The Markdown sources of the PDFs. Same content. Easier to search and edit

2. Every file, and whether you need it to deploy

The product itself (needed to run)

File	What it is	Needed in the deployed image
`server/index.js`	Web server: serves the pages, runs the live connections, rate limits, health check	**Yes**
`server/meeting.js`	The meeting rules: prompts, engagement, recall check, agenda, coach summary, games, privacy rules	**Yes**
`public/index.html`	Landing page: create or join a meeting	**Yes**
`public/host.html`	Host console page	**Yes**
`public/join.html`	Attendee page	**Yes**
`public/privacy.html`	Plain-language privacy page	**Yes**
`public/favicon.svg`	Browser tab icon	**Yes**
`public/css/styles.css`	All styling, light and dark themes	**Yes**
`public/js/common.js`	Shared helpers, connection with automatic reconnect	**Yes**
`public/js/index.js`	Landing page behaviour	**Yes**
`public/js/host.js`	Host console behaviour	**Yes**
`public/js/attendee.js`	Attendee behaviour, presenting mode	**Yes**
`public/js/mini.js`	The floating mini window	**Yes**
`public/js/views.js`	Prompt and result display shared by the page and the mini window	**Yes**
`public/js/chart.js`	The engagement chart	**Yes**
`package.json`, `package-lock.json`	Lists the one runtime dependency (`ws`) and the scripts	**Yes**

The minimum to run Engage is these 17 files plus `npm ci`. Nothing else is needed at runtime.

Deployment files (needed to deploy, not part of the running app)

File	What it is
`Dockerfile`	Recipe for the container image. Copies only `package*.json`, `server/` and `public/`
`.dockerignore`	Keeps docs, tests and git history out of the image
`render.yaml`	Blueprint that tells Render how to run Engage (Docker, health check on `/healthz`, settings)

Checking files (needed to verify, not deployed)

File	What it is
`scripts/smoke.mjs`	**The "is it working" test.** Run it against your deployed address. 31 checks
`test/meeting.test.js`, `test/server.test.js`	38 automated tests of the rules and the live connections
`eslint.config.js`	Code style and error checks
`.github/workflows/ci.yml`	Runs the checks automatically on GitHub for every change
`.gitignore`	Files git should ignore

Documents (read, not deployed)

Folder or file	Contents
`docs/*.pdf` and `docs/*/*.pdf`	The five PDFs listed in section 1
`docs/*.md`, `docs/architecture/*.md`	Markdown sources and guides
`docs/design/prototype/index.html`	The clickable prototype
`docs/design/ux-ui-design.html`, `docs/report/index.html`	HTML sources used to make two of the PDFs
`docs/design/img/`, `docs/report/img/`, `docs/architecture/diagrams/`	Pictures used inside the PDFs. You do not need to open them

---

3. Before you deploy

You need

- [ ] A GitHub account with access to the `mrig003/hum` repository. The code is on the branch `claude/practical-lovelace-u4w13f`.
- [ ] A Render account (render.com). Any host that runs Docker containers also works (see "Other hosts" in step 4).
- [ ] A computer with **Node.js 20 or newer** (to run the checks). Check with `node -v`.
- [ ] Optional: a custom domain, and a feedback form (for example a Google Form) whose link you will paste into `ENGAGE_SURVEY_URL`.
- [ ] A person who will be on call during the first pilot meetings.

Important facts about this version

1. **Run exactly one copy.** Meetings live in the server's memory.
2. **A restart or deploy ends every live meeting.** Deploy only when no meeting is running.
3. **Meeting data is deleted 24 hours after a meeting ends.** There is no database.
4. **There are no host accounts.** The host console is reached from the browser that created the meeting, or through the "Copy private host link" button.

---

4. Deploy, step by step

Step 0. Prove it works on your own computer first (10 minutes)

```
git clone https://github.com/mrig003/hum.git
cd hum
git checkout claude/practical-lovelace-u4w13f
npm install
npm run check          # lint and 38 tests. Expect: "# pass 38", "# fail 0"
npm start              # leave this running. Expect: Engage running on http://localhost:3000
```

In a second terminal:

```
npm run smoke -- http://localhost:3000     # Expect: 31 of 31 checks passed. ALL CHECKS PASSED.
```

Then open http://localhost:3000 in a browser and do the manual check in section 5B. **If anything fails here, do not deploy yet.**

Step 1. Put the code where Render can see it

The code is on GitHub on the branch `claude/practical-lovelace-u4w13f`. Choose one:

- **Simple:** point Render at that branch (step 2).
- **Cleaner:** on GitHub, open a pull request from that branch into your main branch, merge it, and point Render at `main`.

Step 2. Create the service on Render

1. In Render, choose **New**, then **Blueprint**.
2. Connect your GitHub account and select the `hum` repository, then choose the branch from step 1.
3. Render reads `render.yaml` and shows one service called `engage`. Choose **Apply**.
4. When asked for `ENGAGE_SURVEY_URL`, paste your feedback form link, or leave it empty.
5. Wait for the first build to finish (a few minutes). The service shows **Live** when the health check passes.

Settings in `render.yaml` (change them in the Render dashboard if you prefer):

Setting	Value	Why
Runtime	Docker	Uses the `Dockerfile`
Health check path	`/healthz`	Render restarts the service if this stops answering
`TRUST_PROXY`	`1`	**Required.** Without it, every visitor looks like one address and the limit of 30 new meetings per hour applies to everyone together
`ENGAGE_SURVEY_URL`	your form link	Shown on the host summary and attendee recap
Plan	`starter`	A paid plan that does not sleep. Check the current price on Render. **Free plans go to sleep, which would break meetings**
Instances	**1**	Do not scale to more than one

> **Not tested.** The blueprint was written from Render's documented format but has not been run on Render. If Render rejects it, create the service by hand: **New, Web Service**, connect the repo, **Runtime: Docker**, **Health Check Path:** `/healthz`, add the two settings above, **1 instance**.

Step 3. Check the address Render gave you

Render gives an address like `https://engage-xxxx.onrender.com`. Open `https://YOUR-ADDRESS/healthz`. You must see:

```
{"ok":true}
```

Step 4. Run the smoke test against the live address

On your computer, in the `hum` folder:

```
npm run smoke -- https://YOUR-ADDRESS
```

You must see `31 of 31 checks passed.` and `ALL CHECKS PASSED. The server is working.` This test creates one throwaway meeting, so it counts toward the limit of 30 new meetings per hour.

Step 5. Do the manual check

Follow section 5B on the live address, using two different browsers.

Step 6. Add a custom domain (optional)

In Render: **Settings, Custom Domains**, add your domain and create the DNS record Render shows. HTTPS is automatic. Run step 4 again with the new address.

Step 7. Set up monitoring

1. Use any uptime service (for example UptimeRobot) to check `https://YOUR-ADDRESS/healthz` every minute. Alert a chat channel or email.
2. Note where the logs are: Render, your service, **Logs**.
3. Agree who watches during pilot meetings.

Other hosts (any Docker host)

```
docker build -t engage .
docker run -p 3000:3000 -e TRUST_PROXY=1 -e ENGAGE_SURVEY_URL=https://your-form engage
```

Put it behind HTTPS (most platforms do this for you), allow **WebSocket** connections, route traffic to port 3000, and run **one** copy. The image has a built-in health check. The container image has not been built in this environment (no Docker engine was running), but the exact files the Dockerfile copies were started and checked.

---

5. How to check that it works

A. Automatic checks

Command	What it checks	You should see
`npm run check`	Code style (lint) and 38 tests of the rules, the live connections, pages and security headers	`# pass 38` and `# fail 0`, no lint errors
`npm run smoke -- URL`	A real meeting against your live address: pages, security headers, create a meeting, host and attendee over a live connection, question, recall check with "Missed that", Focus, end, summary, coach summary, recap, refusing late joins	`31 of 31 checks passed`

The smoke test lines, and what a failure means

Line	If it says FAIL
Health check answers	The server is down or the address is wrong
Page or file loads	The image is missing files. Rebuild from the right branch
Security headers are set	A proxy is removing headers. Ask your host
Meeting can be created (429)	Rate limit hit. Wait an hour, or `TRUST_PROXY` is missing
Host connects over WebSocket	The host blocks WebSockets. Choose a host that allows them
Update arrives in under 1.5 seconds	Slow network or an overloaded small server
Joining after the meeting ended is refused	A rule is broken. Do not use this version. Tell an engineer

B. The 10 minute manual check (two browsers)

Use Chrome (or Edge) for the host, and a second browser such as Safari or Firefox plus a private Chrome window for attendees.

#	Do this	You should see
1	Open the address. Enter a title, an agenda such as `Intro, 5` and `Budget, 10` (one per line), and create the meeting	The host console. The agenda appears in the **Agenda** card
2	Click **Copy chat message** and open the link in a private window. Enter a first name and join	The join screen first lists what the host can and cannot see. Then the live view. The host shows **Connected now: 1**
3	In the attendee window, click **Agree**, then ask a question	Host: reaction count goes up, the question appears with your name
4	Ask a second question with **Post anonymously** ticked	Host sees the question as **Anonymous**, not your name
5	Host: open the **Recall check** tab. Type a question, three options, mark one correct, click **Launch**	Attendee sees the options plus a dashed **Missed that** button. A countdown shows
6	Attendee clicks **Missed that**. Host clicks **Close and show results**	Host: a **Missed that** bar. Attendee: group split and the line "You said you missed it"
7	Host: launch another recall check right away	A message says the next check is available in a time. The button becomes **Launch anyway**
8	Attendee: click **Mini window** (Chrome or Edge)	A small window opens. Click it to expand: a note warns about sharing the entire screen
9	Host launches a poll while the mini window is small	The small window only gets a dot. It does not open by itself
10	Host launches a recall check	The small window **does** open by itself for about 20 seconds
11	Attendee: turn on **I'm presenting**	The mini window closes. A banner says prompts are skipped, none queued. Host launches a poll: the count goes up
12	Turn presenting off. Host clicks **Focus moment**	Attendee sees a dark banner. Answering is blocked. Host clicks **Resume everyone**
13	In Safari or Firefox, click **Mini window**	A small pop-up window opens instead (it does not float above other windows). If blocked, allow pop-ups
14	Host: click **Copy private host link**, open it in another browser	The host console opens. The address bar no longer shows the secret
15	Host: **End meeting**	The summary: coach summary, engagement by topic, results, top questions. The attendee sees a recap. The survey link shows if you set one
16	Open `/privacy.html`	The privacy page loads

C. Check for hidden errors in the browser

1. On the host page, press **F12** (or right-click, **Inspect**), open **Console**. It should have **no red errors**.
2. Open **Network**, filter **WS**. You should see one connection to `/ws` with status **101**.
3. Open **Network**, filter by **Fetch/XHR**. No requests should be red.

D. Check the server

- In Render, **Logs**: on a healthy start you see `Engage running on http://localhost:PORT`.
- Watch for the text `Unhandled message error`. It means a bug. Copy the lines and send them to an engineer.
- In Render, **Metrics**: memory should stay flat during a meeting.

---

6. If something is wrong

What you see	Likely cause	What to do
"Connection lost. Reconnecting" keeps appearing	The network or host is blocking WebSockets, or the server restarted	Test on another network. Confirm the host supports WebSockets. Check Logs for restarts
Meeting vanished, "Meeting not found"	The server restarted or deployed. Meetings are in memory	Create a new meeting. Avoid deploys during meetings. Persistence is the next build
"Too many meetings created"	The limit of 30 per hour per address. Usually `TRUST_PROXY` is not set	Set `TRUST_PROXY=1` and redeploy
Host page says this browser is not the host	The secret is missing from this browser	Open the meeting using **Copy private host link** from the original host browser
"The browser blocked the mini window"	Pop-up blocked	Allow pop-ups for the site and click again
Mini window is not floating over other windows	The browser has no picture-in-picture support (Safari, Firefox)	Expected. It opens as a small pop-up instead. Use Chrome or Edge for the floating version
Copy buttons say "Copy failed"	The page is not on HTTPS	Use the HTTPS address
Service shows "failed" on Render	Build or health check problem	Open the build log. Confirm the branch has the `Dockerfile` and `package-lock.json`
Page is blank, console shows errors about blocked content	A proxy or extension is changing headers	Test in a clean private window. Ask your host about custom headers
Everything is slow	The plan is too small, or too many attendees	Check Metrics. The design target is 200 people per meeting. This has not been load tested

---

7. What this deployment does not give you

- **No saved meetings.** A restart or deploy ends live meetings. Meeting data is dropped 24 hours after it ends.
- **One copy only.** It cannot be made redundant yet.
- **No accounts.** No sign-in, no host history.
- **No delete button.** Data cannot be deleted on request before the 24 hour limit.
- **Not load tested.** The 200 person target is an estimate.
- **Floating mini window in Chrome and Edge is untested in a real window.** The pop-up version is tested.
- **The Render blueprint, Docker image and GitHub workflow have not been run** in this environment. Steps 2 to 4 above will tell you.
- **No formal accessibility audit.** Contrast was measured and three defects were fixed.

The plan for fixing these, in order, is in `docs/architecture/Engage-Engineering-Plan.pdf`. The next build is saving meetings in a database.

---

8. Command reference

Command	What it does
`npm install`	Installs everything needed
`npm start`	Runs Engage on http://localhost:3000
`npm run check`	Lint and all 38 tests
`npm test`	Tests only
`npm run lint`	Style and error check only
`npm run smoke -- https://URL`	The live "is it working" test, 31 checks
`PORT=4000 npm start`	Runs on another port
`ENGAGE_SAMPLE_MS=2000 npm start`	Makes the engagement chart move quickly, for demos